Unsupervised Learning Laboratories

Lab 5

Alex Rodriguez, Michele Alessi

Fall Semester 2026

5 Recap on Dimensionality Reduction Techniques

In this lab we put together all the dimensionality reduction techniques seen so far (PCA, Kernel PCA, Isomap) and we add three new ones: t-SNE, UMAP and the autoencoder. All of them are applied to the same dataset, so that their results can be compared.

You are required to use **your own implementations** of PCA, Kernel PCA and Isomap from the previous labs, and to write **your own autoencoder** using the layers of `pytorch` (`torch.nn`). For t-SNE and UMAP you can use external libraries (`sklearn`, `umap-learn`).

A starter notebook, `Notebooks/Lab5-DimensionalityReduction.ipynb`, is provided: it downloads and preprocesses the dataset, so that you can start working on the algorithms right away. You are not supposed to modify the preprocessing.

5.1 Exercise 1

The `Fashion-MNIST` dataset contains 28×28 grayscale images of clothes, in 10 classes. The starter notebook takes care of the download, of a random subsample (5000 images for training, 1000 for testing), of the rescaling to $[0, 1]$, of the centering and of the flattening of the images in vectors of 784 pixels.

Apply the following techniques to the training set, plotting the first two (and three) components of the result, coloured by class.

- **PCA** (your own implementation). Plot the eigenvalue spectrum, and show the first principal components as images.
- **Kernel PCA** (your own implementation), with a Gaussian kernel and with a polynomial kernel $k(x, x') = (1 + x^T x')^\delta$. Try different values of σ and of δ.
- **Isomap** (your own implementation), trying different values of the number of neighbours k.
- **t-SNE**: have a look at the documentation of `sklearn.manifold.TSNE`, choose the parameters accordingly, and study the effect of the *perplexity*.
- **UMAP**: have a look at the documentation of `umap-learn`, choose the parameters accordingly, and study the effect of `n_neighbors` and `min_dist`.
- **Autoencoder**: write it with `pytorch` layers, with a bottleneck of dimension 2, and train it with the mean squared error loss, keeping a validation set to monitor the training. Modify the number of layers and the activation functions, and plot the latent space.

Goal. Play with the parameters of these techniques and find the best *hyperparameter–method* combination for the problem at hand: choose (and justify) a quantitative criterion, e.g. the accuracy of a k-nearest-neighbours classifier in the low-dimensional space, or the `trustworthiness` score of `sklearn.manifold`. Discuss the differences between the results.

Hint: Isomap on 5000 points: Floyd-Warshall in pure `numpy` is too slow ($O(n^3)$). You can run your Floyd-Warshall on an undersampled dataset, or use Dijkstra's algorithm (`scipy.sparse.csgraph`) on the whole training set.

Note

Modify the autoencoder such that both the encoder and the decoder have a single layer and a linear activation function. What do you notice? Do you recognize a known structure?

Bonus

Which of these methods can map *new* points (the test set) in the low-dimensional space, without being fitted again? Do it for those that can, and for the others explain why they cannot.